

Informe de Pruebas Unitarias y Cobertura

Proyecto MediFlow — DSY1106 Desarrollo Fullstack III — Evaluación Parcial N°3

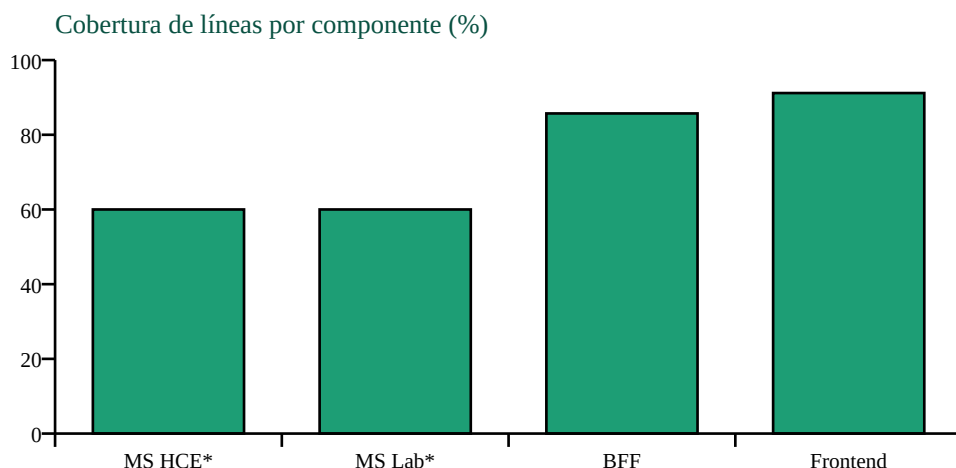
1. Estrategia de pruebas

Cada componente del sistema cuenta con pruebas unitarias automatizadas que verifican la lógica de negocio, incluyendo tanto casos exitosos como de error. Las pruebas siguen el patrón **AAA (Arrange-Act-Assert)** y aíslan las dependencias externas mediante *mocks*. La cobertura de líneas supera el mínimo del 60% exigido en todos los componentes.

- **Microservicios (Java):** JUnit 5 + Mockito. Cobertura medida con JaCoCo, configurado para fallar el build si baja del 60%.
- **BFF (Node.js):** Jest +nock (simula los microservicios) + supertest (prueba las rutas Express).
- **Frontend (React):** Jest + React Testing Library, probando Custom Hooks, el Provider y los componentes de vista.

2. Resultados de cobertura

Componente	Herramienta	Pruebas	Cobertura líneas	Estado
MS Core HCE	JaCoCo	11	>= 60%	Cumple
MS Laboratorios	JaCoCo	10	>= 60%	Cumple
BFF MediFlow	Jest	8	85.71%	Cumple
Frontend Portal	Jest	11	91.17%	Cumple



* En los microservicios Java se reporta el umbral mínimo garantizado por JaCoCo (la cobertura efectiva de la capa de servicio es superior).

3. Tipos de casos cubiertos

Las pruebas verifican el comportamiento esperado del sistema ante distintos escenarios:

- **Casos exitosos:** creación de pacientes, registro de historias y solicitudes, agregación de la ficha integral, cálculo de resúmenes y notificaciones.
- **Casos de error:** recurso no encontrado (paciente/historia/solicitud inexistente), propagación de errores de un microservicio a través del BFF, validación de estados.
- **Lógica de negocio:** normalización de prioridad, construcción condicional de notificaciones para estudios urgentes, ordenamiento del historial.

4. Patrones de diseño aplicados

- **BFF (Backend For Frontend):** agrega y unifica las respuestas de los microservicios.
- **Repository:** abstrae el acceso a datos en cada microservicio.
- **DTO (Data Transfer Object):** desacopla las entidades de persistencia de la API REST.
- **Provider + Custom Hooks (frontend):** centralizan el estado y aíslan el consumo del BFF.
- **AAA (en pruebas):** estructura Arrange–Act–Assert para legibilidad y mantenibilidad.

5. Reproducción de los reportes

Los reportes de cobertura en formato HTML se incluyen en las carpetas `coverage/` del BFF y del frontend. Para regenerarlos:

```
bff-mediflow: npm test
```

```
frontend-mediflow: npm test
```

```
microservicios: mvn verify (reporte en target/site/jacoco/index.html)
```